Dossier IOT401 — Chaîne de supervision IoT sécurisée

Cas d'usage : Ville / Parking — École 2600

Axel Pineau · Clovis Le Floc'h

1er novembre 2026

1. Cahier des charges (CDC)

1.1 Contexte et cas d'usage

Scénario : Notre ville possède deux parkings fermés d'une quarantaine de places, ainsi que 4 capteurs de fumée par parking et trois stations météo mesurant le vent, la température et l'humidité. Les données remontées par ces installations sont publiques pour les citoyens, disponibles sur une application/un site web (tableau de bord). La ville possède également trois radars de vitesse et trois radars de feux rouges, à des fins privées et gouvernementales.

Priorité : Notre ville présente deux priorités de sécurité selon la nature du flux. D'une part, l'intégrité et la détection d'anomalie s'appliquent à tout flux à conséquence légale ou physique directe : les radars (vitesse, feux), dont une mesure falsifiée engage la responsabilité juridique de la commune ou la sécurité d'un usager, et les capteurs de fumée des parkings, dont une panne ou une falsification retarderait une alerte incendie. D'autre part, la résilience s'applique aux flux à impact limité au confort d'usage : occupation des parkings et données météo, dont une interruption ne dégrade que le service rendu aux citoyens, sans risque pour les personnes ni portée légale. Cette double priorité oriente notre conception vers une vérification d'intégrité renforcée sur les flux critiques, et une simple continuité de service sur les flux de confort.

Spécificité : La spécificité de notre cas se traduit par une segmentation des topics MQTT en deux niveaux. Un topic public, exposé au tableau de bord citoyen, diffuse l'occupation des parkings et les données météo. Un topic restreint, interne, porte les données radar (vitesse, feux), à usage privé et gouvernemental.

1.2 Objets et contraintes

Objet	Mesure / rôle	Contrainte(s) associée(s)
Capteur de présence (parking)	Détecte l'occupation d'une place (binaire libre/occupé)	80 unités (2 parkings × 40 places) ; alimentation secteur (infrastructure fixe) ; portée courte intra-parking
Capteur de fumée (parking)	Détecte un départ de feu dans le parking	8 unités (4/parking) ; criticité sécurité des personnes ; latence d'alerte quasi temps réel exigée
Capteur de vent	Mesure environnementale extérieure	3 unités ; alimentation solaire (extérieur)
Capteur de température	Mesure environnementale extérieure	3 unités ; alimentation solaire (extérieur)
Capteur d'humidité	Mesure environnementale extérieure	3 unités ; alimentation solaire (extérieur)
Radar de vitesse	Mesure la vitesse d'un véhicule	3 unités ; portée voirie ; fiabilité critique (valeur à portée légale)
Radar de feu (boucle inductive)	Détecte le franchissement au rouge	3 unités ; criticité légale ; alimentation secteur
Objet intrus (hors allowlist)	Simule un objet non autorisé tentant de se connecter, pour preuve de détection d'intrusion	2 unités (une par broker)

Échelle visée → À l'échelle réelle, l'infrastructure vise 105 objets (80 présence + 8 fumée + 9 météo + 3 radars vitesse + 3 radars feux + 2 intrus pour audit). Le prototype Docker Compose n'en simule qu'un échantillon représentatif par catégorie (2 capteurs de présence, 1 capteur de fumée, 1 station météo complète, 1 radar vitesse, 1 radar feu, 2 objets intrus), le mécanisme d'identité par objet et d'ACL étant indépendant du nombre d'instances et reproductible à l'échelle réelle par simple duplication du service Docker.

Contrainte énergie / alimentation → Les capteurs de présence et de fumée, situés dans des parkings fermés avec infrastructure électrique fixe, sont alimentés sur secteur. Les radars (vitesse, feux), installés en voirie sur mâts dédiés, sont également alimentés sur secteur. Les stations météo, placées en extérieur sans raccordement systématique, fonctionnent sur alimentation solaire avec batterie tampon, ce qui impose une fréquence de reporting modérée pour préserver l'autonomie.

Contrainte réglementaire → Les données de présence (parking) et météo sont des mesures agrégées ou environnementales, non rattachables à un individu : elles peuvent être diffusées publiquement sans restriction RGPD. À l'inverse, les radars de vitesse et de feux rouges peuvent capturer une plaque d'immatriculation, donnée à caractère personnel au sens du RGPD : ces flux restent donc strictement confinés au topic restreint, avec minimisation (seule l'infraction constatée est conservée, pas un flux vidéo continu) et une durée de rétention limitée à l'usage légal (contestation d'amende).

1.3 Choix de protocoles

Critère	Option envisagée 1	Option envisagée 2	Option envisagé 3
Protocole	LoRaWAN	WIFI	Ethernet
Portée	2–5 km en zone urbaine, jusqu'à 15 km en zone dégagée	30–100 m en intérieur	Dépend du câblage (quelques mètres à plusieurs centaines de mètres)
Débit	Très faible : ~0,3 à 50 kbps	Élevé : plusieurs centaines de Mbps	Très élevé : 100 Mbps à plusieurs Gbps
Énergie / autonomie	Très faible conso, plusieurs années sur pile	Conso élevée, alimentation secteur continue requise	Conso élevée, alimentation secteur continue requise
Coût / échelle (milliers d'objets)	Faible coût par objet, une seule passerelle peut couvrir des milliers de capteurs dispersés	Coût modéré par objet, mais forte densité de points d'accès nécessaire pour couvrir toute la ville	Coût par objet faible, mais câblage coûteux à grande échelle ; adapté aux sites fixes déjà câblés (parking, mât radar)
Sécurité native	Chiffrement AES-128, clés réseau et applicatives distinctes (LoRaWAN 1.1)	WPA2/WPA3 au niveau liaison, pas de chiffrement applicatif de bout en bout	Aucune, tout repose sur les couches supérieures (mTLS indispensable)

Protocole réseau retenu → Approche hybride : Ethernet pour les objets fixes (présence, fumée, radars), LoRaWAN pour les stations météo dispersées.

Protocole applicatif retenu → MQTT

Justification de synthèse (2-3 phrases) →Le choix réseau différencié reflète la diversité physique de notre parc d'objets : infrastructure câblée pour les sites fixes à criticité légale (radars) où la fiabilité prime, LoRaWAN pour les capteurs dispersés où l'autonomie et le coût de déploiement à l'échelle priment. MQTT sécurisé par mTLS unifie la couche applicative au-dessus de ces deux réseaux hétérogènes, permettant une authentification et des règles d'accès cohérentes quel que soit le transport physique utilisé.

1.4 Choix des capteurs (références réelles), volumes et coûts

Le prototype simule des capteurs ; pour une ville réelle, nous retenons des références existantes, choisies selon le support de communication et l'alimentation déjà fixés en § 2 et § 3. Les prix sont des prix catalogue relevés le 1er novembre 2026, hors installation, hors TVA sauf mention contraire, et servent à dimensionner le budget (ordre de grandeur, pas devis).

1.4.1 Capteurs retenus

Objet (quantité)	Référence retenue	Pourquoi ce choix	Prix relevé
Station météo : vent, température, humidité (**3 stations**, chacune mesure les 3 grandeurs)	**Milesight WTS506** (LoRaWAN, classe A)	Une seule station mesure vent, température, humidité (plus pression et pluie) : 3 stations au lieu de 9 capteurs séparés. LoRaWAN et panneau solaire 15 W avec 2 batteries de secours (autonomie d'au moins une semaine sans soleil), ce qui correspond à nos contraintes d'extérieur sans secteur. Précisions : ±0,3 °C, ±3 % HR, vent ±0,3 m/s jusqu'à 60 m/s	Prix sur demande (distributeur)
Présence parking (**80**)	**Carlo Gavazzi SBPSUSL45** (ultrasons 40 kHz, une place par capteur)	Conçu pour les places de parking couvert : portée 1,5 m, montage à 2,2-2,4 m, température −40 à 70 °C, LED de statut visible par le conducteur. Alimenté par le bus, adapté au câblage fixe des parkings	**198,98 USD** à l'unité, **165,83 USD** à partir de 63 unités (80 unités ≈ 13 270 USD). Référence indiquée comme arrêtée chez le revendeur : à remplacer par le modèle successeur au moment de l'achat
Radar de vitesse (**3**)	**Stalker II SDR** (radar Doppler directionnel, version stationnaire)	Radar de mesure de vitesse du marché, utilisé pour le contrôle routier ; sert de référence pour dimensionner le coût de la tête de mesure. Un radar de contrôle sanction homologué coûte bien plus et relève d'une homologation métrologique, hors périmètre de l'examen	**1 570 USD** l'unité (grille tarifaire de marché public américain), soit ≈ 4 710 USD pour 3
Radar de feu rouge (**3**)	Boucle inductive + détecteur de boucle **Carlo Gavazzi LD** (sortie relais)	Détection par boucle inductive noyée dans la chaussée, technologie robuste et peu coûteuse, adaptée au franchissement de la ligne pendant la phase rouge (le CDC prévoit une boucle inductive)	**190,77 €** le détecteur (1 boucle), hors pose de la boucle, contrôleur et caméra éventuelle

Objet (quantité)	Référence retenue	Pourquoi ce choix	Prix relevé
Détecteur de fumée (**8**)	Détecteur optique de fumée conforme **EN 54-7** (par exemple gamme AlarmSense 55000-390APO)	Norme européenne obligatoire pour la détection incendie ; détection optique adaptée à un parking couvert ; capteur à contact sec utilisable avec un contrôleur	Prix non relevé : à compléter avec le devis du distributeur

1.4.2 Conséquence sur l'architecture : une passerelle par site

Aucune de ces références ne parle MQTT nativement : le capteur de présence utilise un bus propriétaire (Dupline), le détecteur de boucle et le détecteur de fumée une sortie relais/contact sec, la station météo LoRaWAN. Chaque famille est donc raccordée à un **contrôleur ou une passerelle** qui lit le capteur et publie en **MQTT sur mTLS** sous une identité propre (certificat). Dans notre modèle, l'« objet » qui détient le certificat est cette passerelle ou ce canal ; le prototype le simule par un conteneur par capteur. Un certificat par place de parking serait possible mais lourd : l'alternative réaliste est un certificat par passerelle et par type de flux, ce que l'audit signale comme compromis (un capteur compromis derrière une passerelle partage son identité).

1.4.3 Messages et volumes de données

Hypothèses de dimensionnement retenues pour une ville réelle (ce sont nos choix de conception, pas des valeurs constructeur) :

Objet	Stratégie d'envoi	Messages / jour / objet	Objets	Messages / jour
Présence	changement d'état (≈ 10 par place et par jour) + signal de vie toutes les 5 min	≈ 300	80	≈ 24 000
Fumée	signal de vie chaque minute + alerte immédiate	≈ 1 440	8	≈ 11 520
Météo (3 grandeurs dans un même message)	une trame toutes les 10 min	144	3 stations	432
Radar de vitesse	uniquement les dépassements (≈ 5 % de 3 000 véhicules/jour) + signal de vie toutes les 5 min	≈ 440	3	≈ 1 320
Radar de feu	uniquement les infractions + signal de vie toutes les 5 min	≈ 295	3	≈ 885
Total			**97 objets**	**≈ 38 000 / jour (≈ 0,45 message/s en moyenne)**

À ≈ 100 octets par message (topic `city/sensor/<ID>` d'environ 20 octets, charge utile de 5 à 30 octets, en-têtes MQTT), cela représente **moins de 4 Mo par jour** : très en dessous de la capacité d'un broker Mosquitto, et compatible avec le débit très faible de LoRaWAN pour les stations météo.

Le **signal de vie** est volontaire : sans lui, un capteur de fumée silencieux ressemble à un capteur en panne ou coupé, ce que le superviseur ne peut pas distinguer (risque R8 du rapport d'audit).

Pour comparaison, le **prototype Docker** envoie volontairement plus vite pour la démonstration : présence, température, humidité et vent toutes les 10 secondes (8 640 messages/jour chacun), les capteurs de fumée, de vitesse et de feu seulement sur événement (rare, de l'ordre de quelques messages par heure).

1.4.4 Budget indicatif des têtes de mesure

Poste	Quantité	Prix unitaire	Sous-total
Capteurs de présence	80	165,83 USD (palier ≥ 63)	≈ 13 266 USD
Radars de vitesse	3	1 570 USD	≈ 4 710 USD
Détecteurs de boucle (feu rouge)	3	190,77 €	≈ 572 €
Stations météo	3	sur demande	à compléter
Détecteurs de fumée	8	non relevé	à compléter

Les sous-totaux sont donnés dans la devise relevée, sans conversion. Ils ne comprennent ni l'installation (pose des boucles, câblage, mâts), ni les passerelles, ni le serveur, ni les licences.

Sources : Milesight WTS506 (fiche) · WTS506 chez DistrIoT · Carlo Gavazzi SBPSUSL45 chez e-sonic · fiche SBPSUSL45 · détecteur de boucle LD chez Novapart · grille tarifaire Applied Concepts (Stalker) · détecteur de fumée AlarmSense EN 54-7

2. Architecture et limites de confiance

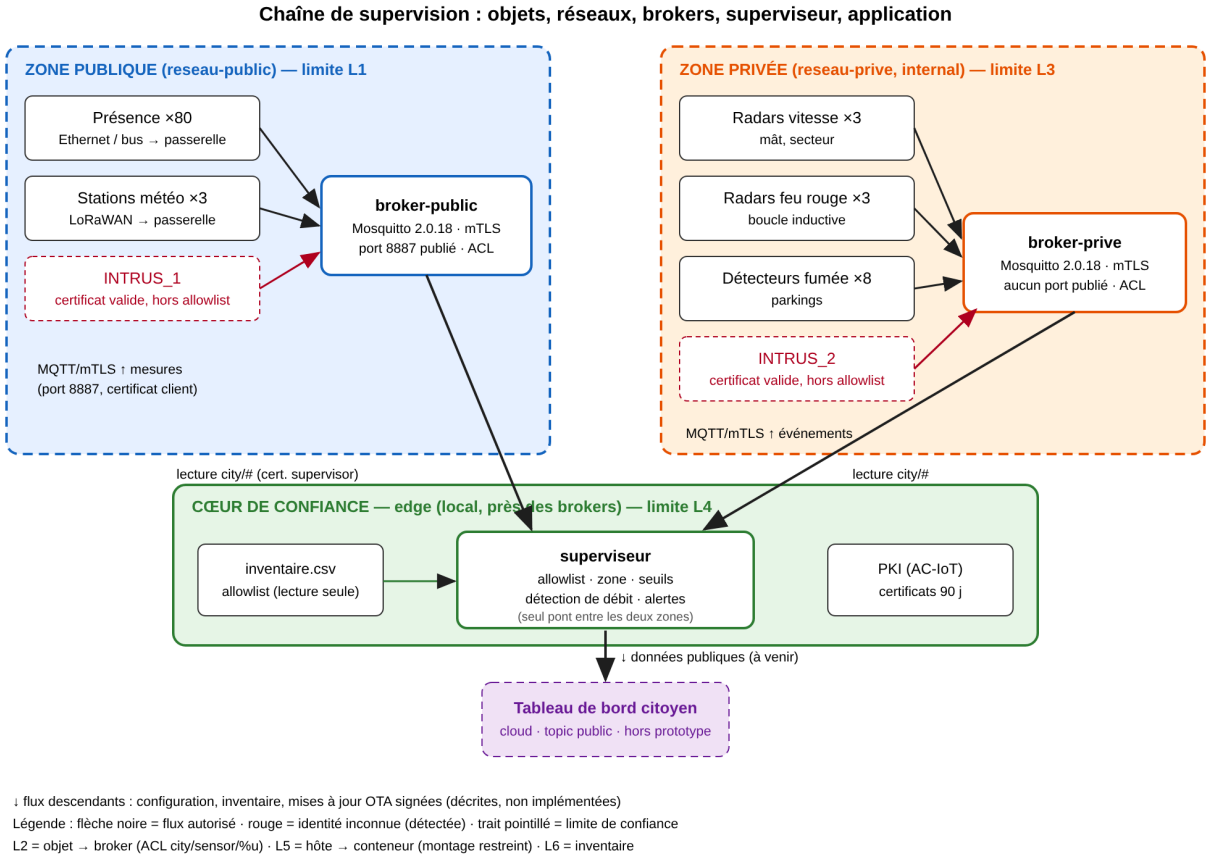


Schéma de la chaîne : objets, réseaux, brokers, superviseur, application, avec les limites de confiance

2.1 Les composants

Couche	Composant	Rôle
Objet	capteurs, radars, détecteurs (via passerelle pour les références réelles du § 1.4)	mesurent et publient sur `city/sensor/<ID>`
Réseau	Ethernet pour les sites fixes, LoRaWAN pour les stations météo, deux réseaux Docker dans le prototype	acheminent les messages
Broker	`broker-public` (port 8887 publié) et `broker-prive` (aucun port publié), Mosquitto 2.0.18	authentifient chaque objet (mTLS), appliquent les ACL
Plateforme	superviseur Python, `inventaire.csv`, PKI interne	contrôle l'identité, la zone, le contenu et le débit, et émet les alertes
Application	tableau de bord citoyen	affiche les données publiques ; **hors prototype**

2.2 Flux de données

- **Flux montants** : chaque objet publie sa mesure ou son événement en MQTT sur mTLS vers le broker de sa zone (QoS 1) ; le superviseur s'abonne à `city/sensor/+` sur les deux brokers.
- **Flux descendants** : configuration, inventaire autorisé et mises à jour logicielles signées vers les objets. Dans le prototype, seul l'inventaire descend (fichier lu par le superviseur) ; l'OTA est décrit au § 3.7 mais non implémenté.
- **Vers l'application** : seules les données du topic public (présence, météo) sont destinées au tableau de bord citoyen, jamais les flux radar (RGPD, § 5).

2.3 Limites de confiance

Limite	Ce qui la franchit	Contrôle
L1 Extérieur → `broker-public`	connexions MQTT sur le port 8887	mTLS (`require_certificate true`), TLS 1.2 minimum, pas d'accès anonyme

Limite	Ce qui la franchit	Contrôle
L2 Objet → broker	publications	identité = CN du certificat ; ACL `pattern write city/sensor/%u` : un objet ne publie que sous son identité
L3 Zone publique ↔ zone privée	aucun flux direct ; uniquement via le superviseur	`reseau-prive` en `internal: true`, aucun port publié sur `broker-prive`
L4 Brokers → superviseur	lecture de `city/#`	certificat `supervisor` + ACL `user supervisor` en lecture seule
L5 Hôte → conteneurs	fichiers de clés et de certificats	chaque service ne monte que ses propres fichiers, en lecture seule
L6 Opérateur → inventaire	`inventaire.csv`	accès au dépôt et à l'hôte, fichier monté en lecture seule

Zones : l'**extérieur** n'est jamais de confiance ; la **zone publique** est semi-confiante (objets exposés physiquement, port ouvert) ; la **zone privée** (radars, fumée) est isolée du réseau extérieur ; le **cœur** (superviseur, inventaire, PKI) est la seule zone de confiance. Le superviseur est le **seul point de jonction** des deux zones : c'est un choix assumé (il faut bien centraliser la supervision) mais aussi le principal point à protéger (risque R4 du § 4).

2.4 Choix edge / cloud

Le **traitement critique est placé en edge** : brokers, superviseur, inventaire et PKI tournent localement, près des objets (salle technique, centre de supervision de la ville). Justification : (1) une alerte incendie ou une alerte d'intrusion ne doit pas dépendre d'une liaison Internet ; (2) les flux radar contiennent potentiellement des données personnelles et restent dans le périmètre de la commune ; (3) le volume est faible (≈ 38 000 messages par jour, § 1.4), donc aucun besoin de capacité élastique. Seul le **tableau de bord citoyen**, qui n'affiche que des données publiques non personnelles, est un candidat naturel pour le cloud (hébergement et montée en charge côté citoyens) ; il n'a qu'un accès en lecture au topic public.

2.5 Identité et nommage

Le **CN du certificat est l'identité de l'objet** (`TEMPERATURE`, `SMOKE`, `presence_1`…). Mosquitto la reprend comme nom d'utilisateur (`use_identity_as_username true`) et l'ACL l'exploite via `%u`. Le superviseur utilise la même identité, extraite du topic, pour la rapprocher de l'inventaire. Ajouter un objet revient à ajouter une ligne dans `inventaire.csv` et à relancer `gen-certs.sh` : l'architecture est indépendante du nombre d'objets.

3. Sécurité implémentée

Cette section décrit ce qui tourne réellement dans le code rendu ; ce qui est seulement décrit est signalé comme tel.

3.1 Identité des objets et mTLS

- **Un certificat par objet** : `certs/gen-certs.sh` émet un certificat X.509 (RSA 2048, signé par `AC-IoT`) pour chaque ligne de `inventaire.csv`, le superviseur et chaque broker : **14 certificats** pour le prototype. Aucune clé n'est partagée entre objets.
- **Provisioning** : le script s'exécute sur le poste de l'opérateur ; chaque objet reçoit son certificat et sa clé par montage de fichiers dans son conteneur (`docker-compose.yml`). En production : provisioning en usine ou à l'installation, clé générée dans l'élément sécurisé de l'objet.
- **Stockage de la clé** : fichiers en mode 600 (`umask 077`), jamais versionnés (`.gitignore`), jamais copiés dans les images ; un conteneur ne voit que son propre couple certificat/clé.
- **Authentification par le broker** : chaque broker est configuré avec `require_certificate true`, `cafile` (AC-IoT), `use_identity_as_username true` et `allow_anonymous false`. Un client sans certificat valide signé par l'AC est rejeté au handshake. L'objet qui présente un certificat valide est identifié par son CN.

3.2 PKI et cycle de vie des clés

Étape	Mise en œuvre / stratégie
Émission	`gen-certs.sh` : AC auto-signée (`CA:TRUE`, 1095 jours) ; certificats serveurs avec SAN (nom DNS du service) et `serverAuth` ; certificats clients avec `clientAuth` ; `CA:FALSE` pour tous les certificats feuilles. **Implémenté**
Durée de vie	certificats feuilles de **90 jours** pour limiter la fenêtre d'exploitation d'une clé volée. **Implémenté**
Rotation	à 30 jours de l'échéance, nouvelle émission et remplacement ; en production, renouvellement automatisé (EST/ACME interne) sans intervention sur site. **Décrit** ; dans le prototype, régénération par `gen-certs.sh` puis `docker compose up --force-recreate`

Étape	Mise en œuvre / stratégie
Révocation	**décrite, non implémentée** : liste de révocation (`crlfile` de Mosquitto) ou OCSP ; coupure immédiate possible en retirant l'identité de `inventaire.csv` (le superviseur la traite alors comme intrus)
Stockage de l'AC	`ca.key` non versionnée ; en production hors ligne ou dans un HSM
Fin de vie	retrait de l'inventaire, expiration du certificat, destruction de la clé

3.3 Chiffrement

- **En transit** : MQTT sur TLS 1.2 minimum (`tls_version tlsv1.2`), authentification **dans les deux sens** : le broker valide le certificat de l'objet, et l'objet comme le superviseur valident celui du broker (`cert_reqs=ssl.CERT_REQUIRED`, avec `ca_certs`). `tls_insecure_set` n'est jamais activé (la ligne figure en commentaire « interdit » dans chaque capteur).
- **Au repos** : le prototype ne stocke aucune mesure (pas de persistance activée côté broker ni base de données) ; les seules données au repos sont les clés (mode 600, hors dépôt). Le chiffrement du disque est à prévoir pour un stockage réel des infractions radar.

3.4 Segmentation et moindre privilège

- **ACL par topic** (identiques sur les deux brokers) : `pattern write city/sensor/%u` (un objet ne publie que sous son identité) ; `user supervisor` / `topic read city/#` (seul le superviseur lit). **Preuve : refus `RC:135`** (§ 4.6).
- **Isolation réseau** : deux réseaux Docker ; `reseau-prive` est `internal: true`, `broker-prive` n'a aucun port publié ; seul `broker-public` publie le port 8887. **Preuve : `broker-prive` introuvable depuis le réseau public** (§ 4.6).
- **Moindre privilège** : chaque service ne monte que ses fichiers de certificats ; configuration et ACL montées en lecture seule ; les brokers tournent avec l'utilisateur non-root `1883:1883` de l'image (la clé du broker appartient à cet utilisateur). Compromis connu : l'UID 1883 du conteneur correspond à un UID de l'hôte, d'où le `sudo chown` demandé par `gen-certs.sh` et documenté dans le README.

3.5 Détection

Le superviseur (`supervisor/supervisor.py`) applique, pour chaque message reçu :

1. **Débit** : fenêtre glissante de 10 secondes ; au-delà de 5 messages, l'identité est placée en liste noire et une alerte `[ALERTE DEBIT]` est émise (une seule alerte ensuite par identité). Ce contrôle limite l'impact d'un objet bavard ou malveillant sur la supervision.
2. **Identité** : tout identifiant absent de `inventaire.csv` déclenche `[ALERTE INTRUSION]`, y compris s'il possède un certificat valide (`INTRUS_1`, `INTRUS_2`).
3. **Zone** : un objet doit arriver par le broker de sa zone déclarée (`[ALERTE ZONE]` sinon).
4. **Contenu** : extraction de la valeur numérique par expression régulière (tolère `23.9°C`, `55 km/h`) ; charge utile non exploitable : `[PAYLOAD INVALIDE]`.
5. **Règle de panne / seuil** : température > 30, humidité > 90, vent > 80, vitesse > 50 déclenchent `[ALERTE SEUIL]` ; tout message du capteur de fumée déclenche `[ALERTE FUMEE]` ; présence et feu rouge sont journalisés comme événements.

Ordre voulu : l'identité est vérifiée **avant** le contenu, pour qu'un intrus soit signalé comme tel et non comme charge utile invalide.

3.6 Secrets

Aucun secret n'est versionné : `.gitignore` exclut `*.key`, `*.pem`, `*.csr`, `*.srl` et `certs/*.crt` ; l'archive de rendu est produite avec `git archive` (uniquement les fichiers suivis par Git). Aucune clé n'est en dur dans le code ni passée par variable d'environnement : les capteurs reçoivent uniquement leur identifiant (`*_ID`), le nom du broker et son port. Les clés sont générées à l'exécution de `gen-certs.sh` sur le poste qui lance le projet.

3.7 Intégrité et mises à jour (décrit)

Non implémenté dans le prototype (objets simulés), mais prévu pour des capteurs réels :

- **Secure boot** : chaîne de démarrage signée depuis une racine de confiance matérielle ; un firmware non signé n'est pas exécuté.
- **OTA signé** : le firmware est signé par la commune (ou le fabricant) ; l'objet vérifie la signature et l'intégrité avant installation, via un canal TLS authentifié.
- **Anti-rollback** : compteur de version monotone, pour interdire le retour à une version vulnérable.
- **Identité protégée** : clé privée stockée dans un élément sécurisé (TPM/SE), non extractible.

3.8 Lancer l'infrastructure

Voir le `README.md` à la racine du dépôt : `bash certs/gen-certs.sh`, puis `docker compose up -d --build` et `docker compose logs -f supervisor`. Les commandes pour rejouer les preuves (mTLS, `RC:135`, intrusion, segmentation) y figurent aussi.

4. Rapport d’audit de notre infrastructure

4.1 Périmètre et méthode

Nous auditons **notre propre chaîne** telle qu’elle tourne dans le dépôt : capteurs simulés → MQTT sur mTLS (port 8887) → deux brokers Mosquitto 2.0.18 (`broker-public`, `broker-prive`) → un superviseur Python (paho-mqtt 2.1.0). Le prototype simule un échantillon de la flotte visée (105 objets) : 2 capteurs de présence, 1 capteur de fumée, 1 station météo (température, humidité, vent), 1 radar de vitesse, 1 radar de feu et 2 objets intrus.

Méthode : STRIDE appliqué aux flux qui franchissent une limite de confiance, deux arbres d’attaque, puis une table menace → contre-mesure → implémentation, priorisée par impact × probabilité. Les preuves exécutées sont en § 4.6.

Cohérence avec le CDC. Le CDC fixe deux priorités : *intégrité et détection d’anomalie* pour les radars (valeur à portée légale) et la fumée (alerte incendie), *résilience* pour la présence et la météo. Nous priorisons donc les menaces d’abord sur l’atteinte à l’intégrité des flux radar/fumée, puis sur la disponibilité de la supervision.

Zones et limites de confiance **L1 à L6** : voir le schéma et le tableau du § 2.3 (zone publique : `broker-public`, port 8887 ; zone privée : `broker-prive`, réseau `internal` ; cœur : superviseur, inventaire, PKI).

Flux étudiés : **F1** objet public → `broker-public` (L1, L2) ; **F2** objet privé → `broker-prive` (L2, L3) ; **F3** `broker-public` → superviseur (L4) ; **F4** `broker-prive` → superviseur (L3, L4) ; **F5** fichiers PKI → conteneurs (L5) ; **F6** inventaire → superviseur (L6).

4.2 STRIDE par flux

Légende de la colonne *Réf.* : renvoi vers la table du § 4.4.

Flux	S — Usurpation	T — Altération	R — Répudiation	I — Divulgation	D — Déni de service	E — Élévation
F1 objet public → broker	T01 : client sans certificat ou avec certificat d’une autre AC	T03 : modification d’une mesure en transit	T06 : un objet nie un envoi	T08 : écoute de la météo/présence (publiques par conception, mais écoute du canal)	T09 : flood depuis une identité valide ou depuis Internet	T11 : objet public publie/lit hors de son topic
F2 objet privé → broker	T02 : usurpation d’un radar/capteur de fumée	T04 : falsification d’une valeur radar ou fumée	T06	T08 : écoute des flux radar	T09	T11
F3/F4 brokers → superviseur	T12 : faux superviseur qui lit `city/#`	T05 : injection par un intrus (certificat valide, inconnu)	T07 : alerte sans trace exploitable	T13 : lecture de `city/#` par un autre client	T14 : superviseur noyé	T15 : compromission du superviseur (pont des zones)
F5 fichiers PKI	—	T16 : remplacement de `ca.crt`	—	T17 : vol d’une clé privée sur l’hôte ou via un conteneur voisin	—	T18 : conteneur lisant les clés des autres
F6 inventaire	—	T19 : ajout d’un intrus dans l’allowlist	—	—	—	—

Menaces transverses : **T20** injection de caractères dans les logs via la charge utile (le superviseur journalise `{texte}` tel quel) ; **T21** absence de révocation d’un certificat compromis.

4.3 Arbres d’attaque

4.3.1 Objectif A — « Faire accepter par la supervision une fausse mesure radar ou un faux silence du capteur de fumée » (priorité intégrité du CDC)

```
A. Faire accepter une fausse mesure (radar / fumée) par le superviseur
├─ A1. Passer le mTLS sans certificat valide ........................... BLOQUÉ (T01) — preuve 1
├─ A2. Obtenir un certificat valide
│   ├─ A2a. Voler la clé d'un objet réel (SPEED, SMOKE...) ............. RÉSIDUEL (T17)
│   │       → publie sur son propre topic : valeur plausible acceptée
│   ├─ A2b. Voler ca.key et émettre un certificat à CN d'un objet réel . RÉSIDUEL (T16) — R5
│   └─ A2c. Faire émettre un certificat à un nom nouveau ................ DÉTECTÉ (T05) : allowlist —
preuve 3
├─ A3. Avec un certificat valide quelconque : publier sur le topic d'un autre objet
│                                                                    BLOQUÉ (T02, T11) — ACL, RC:135 —
preuve 2
├─ A4. Atteindre le broker privé depuis l'extérieur ..................... BLOQUÉ (L3) : pas de port,
```

```
réseau internal
├─ A5. Passer par un autre message valide (rejeu) ..... NON TRAITÉ (R3) : pas
d'horodatage signé
└─ A6. Compromettre le superviseur ..... RÉSIDUEL (T15) — R4
```

4.3.2 Objectif B — « Aveugler la supervision (masquer un départ de feu ou une panne) »

```
B. Empêcher l'alerte d'arriver à l'exploitant
├─ B1. Saturer le port 8887 depuis Internet ..... RÉSIDUEL (T09) — R1 : pas de
limite par client dans Mosquitto 2.0.18
├─ B2. Flooder avec un certificat valide pour noyer les logs ..... ATTÉNUÉ : détection de débit (>
5 msg / 10 s) puis blacklist, une seule alerte par identité
├─ B3. Faire blacklister un objet légitime (déni ciblé)
│   └─ nécessite sa clé (A2a) car l'ACL interdit de publier sous son identité . RÉSIDUEL
├─ B4. Arrêter le superviseur ou un broker ..... ATTÉNUÉ : `restart` Docker,
reconnexion en boucle ; pas de supervision du superviseur (R7)
└─ B5. Couper le capteur de fumée (physique) ..... HORS PROTOTYPE : pas de
détection d'absence de heartbeat (R8)
```

4.4 Table menace → contre-mesure → implémentation, priorisée

Échelle : impact et probabilité de 1 à 3 ; score = impact × probabilité. Le score est évalué **avant** contre-mesure (risque brut), l'état indique ce qui subsiste.

Réf.	Menace	Imp.	Prob.	Score	Contre-mesure	Où dans le code rendu	État
T01	Connexion sans certificat valide	3	3	**9**	mTLS : `require_certificate true`, `cafile`, `allow_anonymous false`, TLS 1.2	`broker-*/mosquitto.conf`	Traité — preuve 1
T05	Objet inconnu avec certificat valide injecte des mesures	3	3	**9**	allowlist issue de `inventaire.csv`, contrôle d'identité avant tout traitement du contenu	`supervisor.py` (`ALLOWLIST`, `on_message`)	Traité — preuve 3
T02/T11	Un objet publie sur le topic d'un autre (ex. `TEMPERATURE` sur `HUMIDITY`)	3	2	**6**	ACL `pattern write city/sensor/%u` (identité du certificat)	`broker-*/acl`	Traité — preuve 2 (RC:135)
T04	Falsification de valeur radar/fumée par l'objet lui-même (clé volée)	3	2	**6**	seuils de vraisemblance, rotation 90 jours (stratégie) ; clé protégée par droits 600	`supervisor.py` (`SEUILS`), `gen-certs.sh` (`umask 077`, `-days 90`)	Atténué (R2)
T17	Vol de clé privée (hôte, conteneur voisin)	3	2	**6**	montage unitaire `:ro`, `umask 077`, clés brokers en 600 propriété UID 1883, clés exclues de Git	`docker-compose.yml`, `gen-certs.sh`, `.gitignore`	Atténué
T09/T14	Flood (identité valide ou intrus)	3	2	**6**	fenêtre glissante : > 5 messages en 10 s → identité en blacklist, une alerte `[ALERTE DEBIT]`	`supervisor.py` (`ddos`)	Atténué (R1)
T15	Compromission du superviseur (pont des deux zones)	3	1	3	surface minimale (pas de port, pas de service) ; image `python:3.12-slim` ; certificat dédié	`supervisor/Dockerfile`, compose	Accepté (R4)
T16	Vol de `ca.key`	3	1	3	`ca.key` non versionnée, générée sur le poste de l'opérateur ; en production AC hors ligne/HSM	`.gitignore`, `gen-certs.sh`	Accepté (R5)
T12/T13	Faux superviseur / lecteur indu	2	2	4	lecture réservée à `user supervisor`, certificat requis	`broker-*/acl`	Traité
T03/T08	Écoute ou altération du canal	2	2	4	TLS 1.2, validation des certificats des deux côtés, aucun `tls_insecure`	`mosquitto.conf`, `tls_set(... CERT_REQUIRED)` dans capteurs et superviseur	Traité

Réf.	Menace	Imp.	Prob.	Score	Contre-mesure	Où dans le code rendu	État
T07/T06	Répudiation / absence de trace	2	2	4	`log_type all`, `connection_messages true` ; alertes horodatables par `docker compose logs`	`mosquitto.conf`, `supervisor.py` (`log`)	Atténué
T19	Intrus ajouté à l'allowlist	3	1	3	inventaire dans Git (traçabilité des commits), monté `:ro`	compose	Accepté
T20	Injection dans les logs par la charge utile	1	2	2	**non traité** : `texte` journalisé tel quel ; correction : échapper/tronquer avant `log()`	`supervisor.py` (`on_message`)	Résiduel (R6)
T21	Pas de révocation d'un certificat compromis	3	1	3	stratégie : `crlfile` Mosquitto/OCSP ; coupure immédiate par retrait de l'inventaire côté superviseur	— (décrit)	Résiduel (R9)
T18	Conteneur compromis lit les clés des autres	2	1	2	un service = ses seuls fichiers montés	compose	Traité

Lecture de la priorisation. Les deux risques bruts à 9 (T01, T05) sont ceux qui permettent à un inconnu de parler au superviseur ; ils sont traités et prouvés. Viennent ensuite l'usurpation entre objets et les atteintes à la clé d'un objet réel, qui touchent directement la priorité d'intégrité du CDC (radars, fumée).

4.5 Risques résiduels et limites assumées

- **R1** DoS sur le port 8887 : Mosquitto 2.0.18 n'a pas de limite par client ; la détection de débit est faite après réception, côté superviseur (piste : pare-feu en amont, quotas).
- **R2** Vol de clé d'un objet (radar sur mât, station solaire) : objets exposés physiquement (pistes : élément sécurisé, rotation courte, révocation).
- **R3** Rejeu applicatif : pas d'horodatage ni de compteur signé (piste : numéro de séquence et horodatage).
- **R4** Superviseur = pont des deux zones (piste : durcissement, deux superviseurs et agrégation à sens unique).
- **R5** `ca.key` sur le poste de génération (piste : AC hors ligne ou HSM).
- **R6** Journaux non assainis (charge utile écrite telle quelle) ; **R7** pas de supervision du superviseur ; **R8** pas de détection d'absence de mesure (capteur de fumée muet).
- **R9** Pas de révocation CRL/OCSP ; **R10** liste noire en mémoire, définitive jusqu'au redémarrage ; **R11** le contrôle de débit précède l'allowlist : un intrus qui inonde est signalé en `[ALERTE DEBIT]` après ses premiers messages d'intrusion.

4.6 Preuves exécutées

Les trois preuves exigées par le barème sont rejouables depuis la racine du dépôt, une fois l'infrastructure démarrée (`bash certs/gen-certs.sh` puis `docker compose up -d --build`). Le réseau s'appelle `<projet>_reseau-public`, avec `<projet>` = nom du dossier en minuscules (ici `iot401` : vérifier avec `docker network ls`).

Preuve 1 — mTLS : un client sans certificat est rejeté (`preuve-mtls`)

```
docker run --rm --network iot401_reseau-public \
  -v "$PWD/certs/ca.crt:/ca.crt:ro" eclipse-mosquitto:2.0.18 \
  mosquitto_pub -d -h broker-public -p 8887 --cafile /ca.crt \
  -t city/sensor/TEST -m 1
```

Résultat attendu : le client valide bien le serveur (`--cafile`) mais n'a aucun certificat client ; le broker ferme la connexion (`Error: Protocol error`, `reason(1116)` / échec du handshake). **Capture jointe au rendu : `dossier/preuves/preuve-mtls.png`.**

Preuve 2 — ACL : refus `RC:135` sur le topic d'un autre (`preuve-acl-rc135`)

```
docker run --rm --network iot401_reseau-public \
  -v "$PWD/certs/ca.crt:/c/ca.crt:ro" \
  -v "$PWD/certs/TEMPERATURE.crt:/c/c.crt:ro" \
  -v "$PWD/certs/TEMPERATURE.key:/c/c.key:ro" \
  eclipse-mosquitto:2.0.18 mosquitto_pub -V mqttv5 -d -q 1 -h broker-public -p 8887 \
  --cafile /c/ca.crt --cert /c/c.crt --key /c/c.key \
  -t city/sensor/HUMIDITY -m 99
```

Résultat obtenu : la connexion s'établit (`CONNACK (0)`, certificat valide), puis `Warning: Publish 1 failed: Not authorized.` et `PUBACK (Mid: 1, RC:135)` ; le superviseur ne voit aucune mesure 99 pour `HUMIDITY`. Le flag `-V mqttv5` est nécessaire : en MQTT 3.1.1, Mosquitto acquitte une publication refusée sans code d'erreur (`RC:0`) et la jette silencieusement, ce qui rend le refus invisible côté client. Les capteurs du projet, eux, ne voient donc pas leur refus : seul le broker et l'absence de la valeur côté superviseur en témoignent. **Capture jointe au rendu : `dossier/preuves/preuve-acl-rc135.png`.**

Preuve 3 — Alerte d'intrusion : objet inconnu (`preuve-intrusion`)

`sensors-intrus-1` (public) et `sensors-intrus-2` (privé) ont un certificat signé par l'AC mais aucune ligne dans `inventaire.csv`.

```
        docker compose logs supervisor | grep -E "INTRUSION|DEBIT"
```

Sortie attendue :

```
[ALERTE INTRUSION] objet inconnu 'INTRUS_1' via broker public | city/sensor/INTRUS_1 | ...
[ALERTE DEBIT] INTRUS_1,ADD TO BLACKLIST
```

L'intrus passe le mTLS (certificat valide), prouvant que l'authentification seule ne suffit pas, puis est détecté par l'allowlist, puis isolé par la détection de débit. **Capture jointe au rendu : `dossier/preuves/preuve-intrusion.png`.**

Preuve 4 (complément) — Segmentation

```
        docker run --rm --network iot401_reseau-public eclipse-mosquitto:2.0.18 ping -c1 -W2 broker-prive
```

Résultat attendu : `bad address` (aucune résolution) depuis le réseau public : `broker-prive` n'existe que sur `reseau-prive`. **Capture jointe au rendu : `dossier/preuves/preuve-segmentation.png`.**

5. Rapport de conformité

5.1 Positionnement

Le cas d'usage regroupe deux familles de flux : des données **publiques** (occupation des parkings, météo) diffusées à un tableau de bord citoyen, et des données **restreintes** (radars de vitesse et de feu, fumée) à usage gouvernemental. La commune est à la fois exploitante de l'infrastructure et, selon qui conçoit les objets, intégratrice. Ce rapport positionne le prototype livré (broker + capteurs simulés + superviseur) par rapport aux référentiels demandés ; il distingue ce qui est **implémenté**, **décrit** et **non couvert**.

Statuts : **Conforme** (implémenté et prouvé), **Partiel** (implémenté en partie ou décrit seulement), **Non couvert**.

5.2 ETSI EN 303 645

La norme vise l'IoT grand public ; nous l'appliquons par analogie comme référentiel de bonnes pratiques à l'infrastructure urbaine, en retenant les 13 dispositions du § 5.

Disposition	Application dans l'infrastructure	Statut
5.1 Pas de mot de passe universel par défaut	Aucun mot de passe : une identité cryptographique par objet (certificat X.509, CN = identifiant), `allow_anonymous false`. Aucune clé commune à la flotte.	Conforme
5.2 Politique de signalement des vulnérabilités	Pas de point de contact ni de politique de divulgation ; à créer (adresse dédiée, délais de traitement)	Non couvert
5.3 Mises à jour logicielles	OTA signé décrit dans le dossier (signature du firmware, vérification avant installation, anti-rollback) ; non implémenté. Les images conteneurs sont reconstruites par `docker compose up --build`	Partiel
5.4 Stockage sécurisé des paramètres sensibles	Clés privées générées localement (`umask 077`), jamais versionnées (`.gitignore`), montées en lecture seule et uniquement dans le service concerné ; aucun secret dans le README ni dans une variable d'environnement. Pas d'élément sécurisé matériel	Partiel
5.5 Communications sécurisées	MQTT sur TLS 1.2 minimum, mTLS, validation du certificat des deux côtés, aucun `tls_insecure`	Conforme

Disposition	Application dans l'infrastructure	Statut
5.6 Minimiser la surface d'attaque	Un seul port publié (8887), broker privé non exposé (`internal: true`), ACL minimales, pas d'interface d'administration	Conforme
5.7 Intégrité du logiciel	Secure boot décrit (chaîne de confiance, racine matérielle), non implémenté sur le prototype	Partiel
5.8 Protection des données personnelles	Données de présence et météo non personnelles ; radars : minimisation décrite en § 5.5 ; chiffrement du transport	Partiel
5.9 Résilience aux pannes	`restart` Docker, reconnexion du superviseur en boucle, brokers séparés (la panne d'un broker n'arrête pas l'autre zone) ; pas de redondance du superviseur	Partiel
5.10 Examen des données de télémétrie	Le superviseur examine chaque message (identité, zone, contenu, débit) et journalise les alertes	Conforme
5.11 Effacement simple des données	Hors périmètre du prototype (pas de stockage persistant des mesures)	Non couvert
5.12 Facilité d'installation et de maintenance	Reproductible par `bash certs/gen-certs.sh` puis `docker compose up -d --build` ; ajout d'un objet = une ligne d'inventaire + un service	Conforme
5.13 Validation des données d'entrée	Allowlist des identités ; extraction de valeur par expression régulière ; charges utiles non numériques rejetées (`PAYLOAD INVALIDE`)	Conforme

Bilan : 6 conformes, 5 partielles, 2 non couvertes. Les points faibles sont la gestion des vulnérabilités (5.2), les mises à jour et le secure boot (5.3, 5.7), tous liés à la partie matérielle/firmware que le prototype simule.

5.3 OWASP IoT Top 10

Nous retenons la liste 2018 (la plus répandue) ; trois risques sont particulièrement pertinents pour une flotte urbaine dispersée : I3 (interfaces), I4 (mises à jour), I10 (durcissement physique).

Risque OWASP IoT	Pertinence	Couverture
I1 Mots de passe faibles, devinables ou codés en dur	Forte	Aucun mot de passe ; certificat par objet ; pas de clé partagée par la flotte. **Couvert**
I2 Services réseau non sécurisés	Forte	Un seul service exposé, protégé par mTLS ; segmentation public/privé. **Couvert**
I3 Interfaces d'écosystème non sécurisées	Moyenne	Pas d'API web ni d'interface cloud dans le prototype ; le futur tableau de bord citoyen devra être isolé du broker (lecture seule, topic public uniquement). **Partiel**
I4 Absence de mécanisme de mise à jour sécurisé	Forte	OTA signé décrit, non implémenté. **Partiel**
I5 Composants obsolètes ou non sûrs	Moyenne	Versions épinglées (Mosquitto 2.0.18, paho-mqtt 2.1.0, Python 3.12) ; pas de suivi automatique de CVE. **Partiel**
I6 Protection insuffisante de la vie privée	Forte (radars)	Minimisation et rétention définies en § 5 ; le prototype ne transporte aucune plaque. **Partiel**
I7 Transfert et stockage de données non sécurisés	Forte	TLS en transit ; pas de stockage persistant ; clés protégées sur l'hôte. **Couvert**
I8 Gestion des appareils insuffisante	Forte	Inventaire = allowlist ; détection d'objet inconnu ; pas de révocation CRL. **Partiel**
I9 Configuration par défaut non sécurisée	Moyenne	Configuration durcie explicite (anonyme interdit, TLS 1.2, ACL) ; pas de configuration par défaut permissive. **Couvert**
I10 Absence de durcissement physique	Forte (mâts, stations solaires)	Non traité sur le prototype ; élément sécurisé et capteurs de sabotage à prévoir. **Non couvert**

5.4 Cyber Resilience Act (CRA)

Le règlement (UE) 2024/2847 s'applique aux **produits comportant des éléments numériques** mis sur le marché de l'UE. Un capteur, un radar ou une passerelle fabriqués pour la commune entrent dans le champ ; le logiciel interne d'un opérateur non commercialisé est plus nuancé. Nous raisonnons comme si la commune ou son fournisseur était **fabricant** des objets. Les obligations de signalement s'appliquent depuis le 11 septembre 2026 ; l'essentiel des autres obligations s'applique à partir du 11 décembre 2027.

Obligation CRA	Application au produit	Statut
Sécurité dès la conception et par défaut	mTLS, ACL, moindre privilège, pas de mot de passe par défaut, surface minimale	Conforme
Gestion des vulnérabilités	Processus à instaurer : point de contact, correctifs sans délai, tests réguliers ; **SBOM** (liste des composants : Mosquitto, paho-mqtt, Python, images) à tenir à jour	Non couvert
Mises à jour de sécurité	Mises à jour signées et vérifiées, distribuées gratuitement pendant la période de support, avec possibilité de mise à jour automatique ; OTA décrit mais non implémenté	Partiel
Durée de support	Au moins 5 ans, ou la durée de vie attendue si plus courte. Pour des radars et capteurs de voirie dont la vie dépasse 10 ans, nous visons un support aligné sur la durée de vie attendue, annoncé à l'achat	Décrit
Signalement	Vulnérabilité activement exploitée ou incident grave à notifier à l'ENISA/CSIRT national dans des délais courts (alerte précoce sous 24 h)	Non couvert
Documentation technique et marquage CE	Dossier technique : ce dossier + audit en constitue une base	Partiel

5.5 RGPD

5.5.1 Qualification des données

Flux	Donnée	Personnelle ?	Diffusion
Présence parking	état libre/occupé par place	Non (agrégée, non rattachable à un individu)	Publique
Météo	température, humidité, vent	Non	Publique
Fumée	détection d'un départ de feu	Non	Restreinte (sécurité des personnes)
Radar de vitesse	vitesse d'un véhicule ; potentiellement plaque	**Oui** si la plaque est capturée	Restreinte
Radar de feu	franchissement au rouge ; potentiellement plaque	**Oui** si la plaque est capturée	Restreinte

5.5.2 Finalité

- **Présence et météo** : information des citoyens et gestion des parkings.
- **Fumée** : sécurité des personnes et alerte incendie.
- **Radars** : constatation d'infractions routières, finalité unique et définie par la loi ; aucune réutilisation (pas de suivi des déplacements, pas de statistiques nominatives).

Base légale : mission d'intérêt public (art. 6.1.e) pour la commune ; pour les radars, obligation légale encadrée par le code de la route. À valider avec le délégué à la protection des données de la commune.

5.5.3 Minimisation

- Seul le **minimum nécessaire** est remonté : le radar n'envoie au broker que *vitesse mesurée et horodatage* ; une donnée identifiante (plaque) n'est transmise **que lorsqu'une infraction est constatée**, jamais en flux continu.
- La reconnaissance de plaque et le floutage de tout le reste se font à l'**edge**, sur l'objet, avant l'émission.
- Les flux restreints ne transitent que par le topic privé / `broker-prive`, isolé du réseau public et du tableau de bord citoyen.
- Les mesures publiques ne contiennent aucune information individuelle.
- Dans le prototype, aucune plaque n'est simulée. Point d'attention : le superviseur journalise la charge utile d'un objet inconnu ; il faut tronquer ou masquer ce contenu si un objet envoyait une donnée personnelle.

5.5.4 Rétention

Donnée	Durée proposée	Justification
Mesures publiques (présence, météo)	agrégats conservés pour l'historique du tableau de bord ; valeurs brutes quelques jours	aucune donnée personnelle, usage de confort
Fumée	durée utile à l'enquête d'incident, puis suppression	sécurité
Radar : mesure sans infraction	**suppression immédiate** (non conservée)	minimisation

Donnée	Durée proposée	Justification
Radar : infraction constatée	limitée au traitement de l'infraction et au **délai de contestation de l'amende**, puis suppression ou archivage légal	finalité unique
Journaux techniques (connexions, alertes)	durée courte et définie (quelques semaines)	sécurité du système ; contiennent des identifiants d'objets, pas de personnes

Les durées précises sont à confirmer par la commune (DPO) ; l'important est qu'elles soient **limitées, justifiées par la finalité et appliquées automatiquement**.

5.5.5 Autres exigences

Information des personnes (panneaux, mentions), droits d'accès, **analyse d'impact (AIPD)** probablement nécessaire pour des radars (surveillance systématique de la voie publique), registre des traitements, sécurité du traitement (art. 32) assurée en partie par les mesures techniques de ce prototype (mTLS, ACL, segmentation).

6. Limites et améliorations

Section non notée. Le détail des risques (R1 à R11) est au § 4.5 ; ci-dessous ce que le prototype ne couvre pas.

- **Échelle et réalisme** : 10 conteneurs d'objets (8 objets et 2 intrus) sur 105 visés, valeurs simulées ; les références réelles du § 1.4 ne parlent pas MQTT et supposent une passerelle (identité par passerelle ou par canal, non simulée). Comportement en charge non mesuré.
- **Panne silencieuse** : fumée, vitesse et feu rouge ne publient que sur événement ; un capteur muet (panne ou coupure) n'est pas détecté. Piste : signal de vie et alerte « silence ».
- **Déni de service et rejeu** : pas de limite de débit par client dans Mosquitto 2.0.18 (détection côté superviseur, liste noire en mémoire), pas de protection contre le rejeu applicatif. Pistes : limitation en amont, quotas, horodatage et numéro de séquence signés.
- **Révocation** : pas de CRL ni d'OCSP ; coupure par l'inventaire et rotation à 90 jours.
- **Superviseur** : seul pont entre les zones, non redondant ; journaux non assainis (charge utile d'un intrus écrite telle quelle) ; toute publication de fumée déclenche l'alerte sans validation de valeur.
- **Matériel et chaîne de mise à jour** : secure boot, OTA signé et élément sécurisé décrits (§ 3.7), non implémentés ; tableau de bord citoyen hors prototype.
- **Clés** : `ca.key` et les clés générées sur un même poste, `sudo chown` pour les clés des brokers (UID 1883, documenté dans le README). Production : AC hors ligne ou HSM, gestionnaire de secrets.
- **Conformité** : auto-évaluée ; gestion des vulnérabilités (CRA), SBOM et analyse d'impact RGPD restent à mener.

7. Contributions : qui a fait quoi

Partie	Responsable
Arborescence du dépôt, `.gitignore`	Clovis Le Floc'h
`README.md`	Clovis Le Floc'h
`certs/gen-certs.sh` (PKI), fichiers `.ext`, `inventaire.csv`	Clovis Le Floc'h
`broker-public/` et `broker-prive/` (`mosquitto.conf`, `acl`)	Clovis Le Floc'h
`supervisor/supervisor.py` (allowlist, seuils, détection d'intrusion, contrôle de zone)	Clovis Le Floc'h
Détection de débit (rate limiting) du superviseur	Axel Pineau
Tous les `Dockerfile`	Axel Pineau
Scripts de tous les capteurs (`sensors/`)	Axel Pineau
`docker-compose.yml`	Clovis Le Floc'h et Axel Pineau
Cahier des charges (§ 1)	Clovis Le Floc'h et Axel Pineau
Rapport d'audit (§ 4) et rapport de conformité (§ 5), exécution des preuves	Clovis Le Floc'h
Assemblage du dossier (§ 2, 3, 6, 7)	Clovis Le Floc'h

Outil d'assistance : un assistant d'IA générative (Claude) a été utilisé pour la mise au point des commandes, la rédaction des rapports d'audit et de conformité et la relecture du dossier. Le code et les configurations ont été écrits par les membres du binôme, qui peuvent expliquer chacun des fichiers rendus.

Nous attestons l'exactitude de ce tableau.

Clovis Le Floc'h : _____ Axel Pineau : _____

Fait le 1er novembre 2026.